

Azure Service Bus – Sending a Message to a Queue Using C

1. What We Will Learn

In this example, we will learn how to send a message from a **C# Console Application** to an **Azure Service Bus Queue**.

We will use:

1. `ServiceBusClient`
2. `ServiceBusSender`
3. `ServiceBusMessage`

The overall flow is:

```
C# Console Application
    |
    v
ServiceBusClient
    |
    v
ServiceBusSender
    |
    | Send Message
    v
Azure Service Bus Queue
```

After sending the message, we will verify it using **Service Bus Explorer** in the Azure Portal.

2. Prerequisites

Before writing the code, we need:

- An Azure Service Bus Namespace
- A Service Bus Queue
- A Shared Access Policy
- Connection String
- Queue Name
- Visual Studio
- .NET Console Application

3. Install Azure Service Bus NuGet Package

Create a new **Console Application** in Visual Studio.

Example project name:

ServiceBusSender

Then install the required NuGet package:

Azure.Messaging.ServiceBus

Steps

In Visual Studio:

Project
↓
Manage NuGet Packages
↓
Browse
↓
Azure.Messaging.ServiceBus
↓
Install

This package provides the classes required to work with Azure Service Bus from C#.

Important classes include:

ServiceBusClient
ServiceBusSender
ServiceBusReceiver
ServiceBusProcessor
ServiceBusMessage

4. Step 1 – Get the Connection String

We need a connection string to connect our C# application to Azure Service Bus.

In Azure Portal:

```
Azure Service Bus Namespace
    ↓
Queue
    ↓
Settings
    ↓
Shared access policies
    ↓
Open the required policy
    ↓
Copy Connection String
```

The connection string is used to create the `ServiceBusClient`.

Conceptually:

```
Connection String
    ↓
ServiceBusClient
    ↓
Azure Service Bus
```

5. Step 2 – Get the Queue Name

We also need the name of the queue to which we want to send the message.

For example:

```
Queue Name = orders
```

The queue name tells the `ServiceBusSender` where the message should be sent.

6. Step 3 – Create ServiceBusClient

First, define the connection string:

```
string connectionString = "...";
```

Then create the `ServiceBusClient`:

```
ServiceBusClient serviceBusClient =  
    new ServiceBusClient(connectionString);
```

What is ServiceBusClient?

`ServiceBusClient` is the object used to connect the C# application to Azure Service Bus.

```
C# Application  
    |  
    v  
ServiceBusClient  
    |  
    v  
Azure Service Bus
```

7. Step 4 – Create ServiceBusSender

Now create a sender using the `ServiceBusClient`.

```
ServiceBusSender serviceBusSender =  
    serviceBusClient.CreateSender(queueName);
```

Here:

- `serviceBusClient` → connection to Service Bus
- `CreateSender()` → creates a sender
- `queueName` → specifies the destination queue

Flow:

```
ServiceBusClient
  |
  | CreateSender("QueueName")
  v
ServiceBusSender
```

8. Step 5 – Create the Message

Now create a `ServiceBusMessage`.

```
ServiceBusMessage serviceBusMessage =
    new ServiceBusMessage("Hello from C# and Visual Studio");
```

Here:

```
ServiceBusMessage
  |
  v
"Hello from C# and Visual Studio"
```

The message contains the data that we want to send to the queue.

9. Step 6 – Send the Message

Now use the `ServiceBusSender` to send the message.

```
await serviceBusSender.SendMessageAsync(serviceBusMessage);
```

The flow is:

```
C# Application
  |
  v
ServiceBusMessage
  |
  v
ServiceBusSender
```

```
|  
| SendMessageAsync()  
v  
Azure Service Bus Queue
```

10. Complete Basic Code

The complete example looks like this:

```
using Azure.Messaging.ServiceBus;  
  
string connectionString = "<SERVICE_BUS_CONNECTION_STRING>";  
string queueName = "<QUEUE_NAME>";  
  
ServiceBusClient serviceBusClient =  
    new ServiceBusClient(connectionString);  
  
ServiceBusSender serviceBusSender =  
    serviceBusClient.CreateSender(queueName);  
  
ServiceBusMessage serviceBusMessage =  
    new ServiceBusMessage("Hello from C# and Visual Studio");  
  
await serviceBusSender.SendMessageAsync(serviceBusMessage);  
  
Console.WriteLine("Message has been sent");
```

11. Understanding the Code Step-by-Step

Step 1

```
using Azure.Messaging.ServiceBus;
```

Imports the Azure Service Bus classes.

Step 2

```
string connectionString = "<SERVICE_BUS_CONNECTION_STRING>";
```

Stores the connection information required to connect to Azure Service Bus.

Step 3

```
string queueName = "<QUEUE_NAME>";
```

Stores the name of the destination queue.

Step 4

```
ServiceBusClient serviceBusClient =  
    new ServiceBusClient(connectionString);
```

Creates the Service Bus client.

Think:

Client = Connection

Step 5

```
ServiceBusSender serviceBusSender =  
    serviceBusClient.CreateSender(queueName);
```

Creates a sender for the specified queue.

Think:

Sender = Send

Step 6

```
ServiceBusMessage serviceBusMessage =  
    new ServiceBusMessage("Hello from C# and Visual Studio");
```

Creates the message that we want to send.

Think:

Message = Data

Step 7

```
await serviceBusSender.SendMessageAsync(serviceBusMessage);
```

Sends the message asynchronously to the queue.

Think:

SendMessageAsync = Send the message

Step 8

```
Console.WriteLine("Message has been sent");
```

Displays confirmation in the console after the send operation completes successfully.

12. Why Do We Use `await`?

The method:

```
SendMessageAsync()
```

is asynchronous.

Therefore, we use:


```
await
```

Example:

```
await serviceBusSender.SendMessageAsync(serviceBusMessage);
```

This means:

Wait for the asynchronous send operation to complete without blocking the current thread.

If the send succeeds, execution continues to:

```
Console.WriteLine("Message has been sent");
```

13. Verify the Message in Azure Portal

After running the C# application, we can verify whether the message actually reached the queue.

Go to:

```
Azure Portal
  ↓
Service Bus Namespace
  ↓
Queue
  ↓
Service Bus Explorer
```

In Service Bus Explorer, we can inspect the messages available in the queue.

If the queue contains one message, we may see:

```
Active Messages: 1
```

Then select the option to peek/read messages.

The message should contain:

Hello from C# and Visual Studio

This confirms that the C# application successfully sent the message to Azure Service Bus.

14. What is Service Bus Explorer?

Service Bus Explorer is a tool in the Azure Portal that allows us to interact with Service Bus entities and inspect messages.

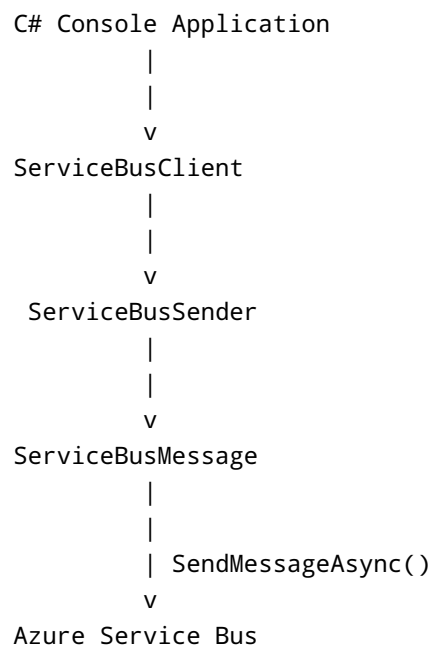
It can be useful for:

- Checking messages
- Peeking messages
- Sending test messages
- Receiving messages
- Troubleshooting messaging issues

In this example, we use it to verify that our C# application successfully sent the message.

15. Complete Message Flow

The entire process can be visualized as:



```

      |
      v
    Queue
      |
      v
Service Bus Explorer
      |
      v
"Hello from C# and Visual Studio"

```

16. Role of Each Class

Class	Purpose
<code>ServiceBusClient</code>	Connects the application to Azure Service Bus
<code>ServiceBusSender</code>	Sends messages
<code>ServiceBusMessage</code>	Represents the message being sent
<code>ServiceBusReceiver</code>	Receives messages
<code>ServiceBusProcessor</code>	Continuously receives and processes messages

For this example, we mainly use:

```

ServiceBusClient
  ↓
ServiceBusSender
  ↓
ServiceBusMessage

```

17. Important Interview Question

How do you send a message to Azure Service Bus using C#?

Answer

First, we install the `Azure.Messaging.ServiceBus` NuGet package. Then we create a `ServiceBusClient` using the Service Bus connection string. From the client, we create a `ServiceBusSender` by providing the queue name. Next, we create a `ServiceBusMessage` containing the data and send it using `SendMessageAsync()`.

Flow:

```
Connection String
    ↓
ServiceBusClient
    ↓
ServiceBusSender
    ↓
ServiceBusMessage
    ↓
SendMessageAsync()
    ↓
Azure Service Bus Queue
```

18. Important Interview Questions

Q1. What NuGet package is required?

```
Azure.Messaging.ServiceBus
```

Q2. What is `ServiceBusClient` ?

It is the client used to establish a connection between the C# application and Azure Service Bus.

Q3. What is `ServiceBusSender` ?

It is used to send messages to a specific Service Bus queue or topic.

Q4. What is `ServiceBusMessage` ?

It represents the message that we want to send through Azure Service Bus.

Q5. How do you specify the destination queue?

Using:

```
serviceBusClient.CreateSender(queueName);
```

The `queueName` specifies the destination queue.

Q6. Which method is used to send a message?

```
await serviceBusSender.SendMessageAsync(serviceBusMessage);
```

Q7. How can you verify that the message was sent?

We can use **Service Bus Explorer** in the Azure Portal to inspect the queue and peek at the messages.

19. Important Security Point

For learning purposes, the connection string may be placed directly in the code.

However, in a real application, **do not hard-code the connection string**.

Avoid:

```
string connectionString =  
    "Endpoint=sb://...;SharedAccessKey=...";
```

in production source code.

Instead, use secure approaches such as:

- Azure Key Vault
 - Managed Identity
 - Secure configuration
 - Environment variables
 - Application configuration/secrets management
-

20. Real-World Example

Suppose we have an Order Service.

When a customer places an order:

```
Customer
  |
  v
Order Service
  |
  | Create Order Message
  v
ServiceBusSender
  |
  v
Order Queue
```

The message could contain:

```
{
  "OrderId": 1001,
  "CustomerId": 501,
  "Amount": 2500
}
```

A Payment Service can later receive and process this message.

```
Order Service
  |
  | Send
  v
Azure Service Bus
  |
  v
Order Queue
  |
  | Receive
  v
Payment Service
```

This is how Azure Service Bus helps us build **loosely coupled microservices**.

21. Quick Revision

Remember this sequence:

```
1. Connection String
   ↓
2. ServiceBusClient
   ↓
3. ServiceBusSender
   ↓
4. ServiceBusMessage
   ↓
5. SendMessageAsync()
   ↓
6. Azure Service Bus Queue
```

Memory Trick

Client → Sender → Message → Send

```
CLIENT  = Connect
SENDER   = Send
MESSAGE  = Data
ASYNC    = Send without blocking the current thread
```

One-Line Interview Answer

To send a message from a C# application to Azure Service Bus, we create a `ServiceBusClient` using the connection string, create a `ServiceBusSender` for the queue, create a `ServiceBusMessage`, and send it using `SendMessageAsync()`.